Project 2 – Graph Database Design and Cypher Query

- **Unit:** CITS5504 Data Warehousing
- **Student Name:** Junjian Long
- **Student Number:** 24702822

---

1. Graph Database Design

1.1 Property Graph Schema

The property graph schema was designed using the Arrows App and is shown in Figure 1 below. The model consists of two node types — **Airport** and **Airline** — and two relationship types — **ROUTE** and **OPERATES**.

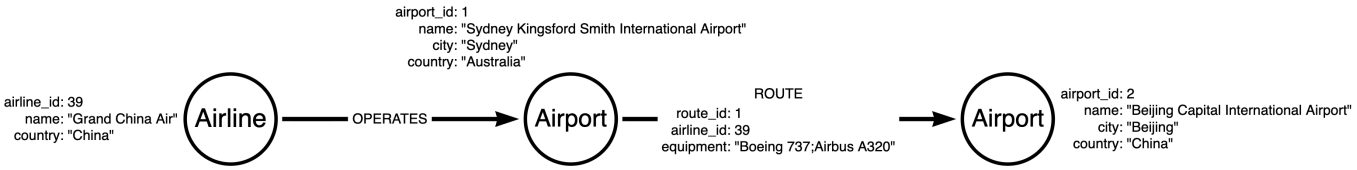

Figure 1. Property graph schema designed in Arrows App.

Node: Airport

Property	Type	Description
`airport_id`	Integer	Unique identifier (auto-assigned during ETL)
`name`	String	Full airport name
`city`	String	City of the airport
`country`	String	Country/region of the airport

Node: Airline

Property	Type	Description
`airline_id`	Integer	Unique identifier (auto-assigned during ETL)
`name`	String	Full airline name
`country`	String	Country of the airline's registration

Relationship: (Airport)-[:ROUTE]->(Airport)

Property	Type	Description
`route_id`	Integer	Unique route record identifier
`airline_id`	Integer	Foreign reference to the operating Airline node
`equipment`	String	Semicolon-separated list of aircraft types (e.g., `"Boeing 737;Airbus A320"`)

Relationship: (Airline)-[:OPERATES]->(Airport)

No properties. Represents the fact that an airline has at least one route departing from the given airport, providing a direct graph edge between Airline and Airport nodes.

1.2 Design Choices and Discussion

1.2.1 ROUTE as a Relationship, Not a Node

The most critical design decision was whether to model a route as a **relationship** or as an intermediary **node**. This model adopts the former approach, representing each service record as a `[:ROUTE]` relationship directed from the departure airport to the arrival airport.

The primary advantage is the native support for variable-length path queries. Neo4j's Cypher syntax allows `[:ROUTE*1..3]` to express "traverse up to three route hops" in a single clause, which directly answers Question 5 of this project (paths from Beijing to Perth). Had routes been modelled as nodes, finding a three-hop path would require alternating node–relationship traversals (Airport→Route→Airport→Route→Airport→Route→Airport), doubling the traversal depth and significantly complicating the query [1].

This aligns with the property graph principle described by Robinson et al., who note that "relationships are first-class citizens" in a graph model and that the choice to use a relationship versus an intermediary node should be guided by whether the connection itself carries semantics independently or only exists to link two entities [2].

The trade-off is that route-level aggregations (e.g., counting all routes for a given airline) require filtering on relationship properties rather than matching node labels. However, Neo4j's relationship-property indexes mitigate this performance concern [3].

1.2.2 Independent Airline Nodes

Airline attributes (`name` , `country`) could have been embedded directly as properties on each `[:ROUTE]` relationship, avoiding the need for a separate `Airline` node. However, this design maintains an independent `Airline` node for two reasons:

1. **Query efficiency for airline-centric queries:** Question 1 (list all airlines from Australia) is answered by a single `MATCH (al:Airline {country: 'Australia'})` without scanning any relationships. Embedding the country in every ROUTE relationship would require scanning all 57,301 relationships.
2. **Single source of truth:** Storing airline attributes once on the node and referencing via `airline_id` on the ROUTE relationship eliminates redundancy. If an airline name were to change, only one node needs to be updated [2].

1.2.3 OPERATES Relationship for Graph Connectivity

Without the `[:OPERATES]` relationship, the `Airline` nodes would be structurally isolated — connected to `ROUTE` relationships only via the `airline_id` property value, not via actual graph edges. This is an anti-pattern in graph databases, as it reduces the graph to a property store rather than a connected network [1].

The `(Airline)-[:OPERATES]->(Airport)` relationship was therefore added to encode "this airline operates at least one route departing from this airport." This relationship carries no properties because it expresses pure existence. It enables idiomatic Cypher queries such as `MATCH (al:Airline)-[:OPERATES]->(ap:Airport)` and ensures the schema visualisation correctly reflects the connected structure of the graph.

1.2.4 Country Stored as a Node Property, Not a Separate Node

An alternative design would represent each country as a dedicated `Country` node, with `Airport` and `Airline` nodes connected to it via a `[:LOCATED_IN]` or `[:REGISTERED_IN]` relationship. This would eliminate the redundancy of storing the same country string across hundreds of airport records and would allow efficient traversal of the form "find all airports in a given country."

However, for this project, `country` is retained as a string property on both `Airport` and `Airline` nodes. This decision is justified by the query requirements: all country-based filters in Q1–Q6 are simple equality checks (`WHERE a.country = 'Australia'`), which are efficiently served by a property index. Introducing `Country` nodes would require an additional relationship hop for every country-based filter without providing traversal benefits, since no query requires navigating from one country to another through the graph. The trade-off — slight data redundancy in exchange for simpler queries — is appropriate for a route-focused analytical workload [1].

1.2.5 Equipment Stored as a Semicolon-Separated String

The original dataset stores aircraft types as a semicolon-delimited string per route record (e.g., `"Boeing 737;Airbus A320;ATR 72"`). Rather than normalising this into separate `AircraftType` nodes, the raw string is preserved on the `[:ROUTE]` relationship's `equipment` property. At query time, Cypher's built-in `split()` function and `UNWIND` clause are used to decompose the string into individual types for counting (Question 4).

This avoids the overhead of additional nodes and relationships for aircraft data, which is not a primary analytical entity in the domain. Angles and Gutierrez note that schema flexibility is a key advantage of property graphs, and property-level storage is appropriate when an attribute is primarily used as a filter or aggregation input rather than a traversal endpoint [4].

1.2.6 Summary of Design Decisions

The table below summarises the five key design decisions, the alternatives considered, and the rationale for each choice:

Decision	Choice Made	Alternative Considered	Rationale
Route modelling	`[:ROUTE]` relationship	Route as a node	Enables `[:ROUTE*1..3]` variable-length path syntax; reduces traversal depth
Airline representation	Dedicated `Airline` node	Inline properties on `[:ROUTE]`	Efficient Q1-style airline-centric queries; single source of truth
OPERATES relationship	Added `(Airline)-[:OPERATES]->(Airport)`	No Airline–Airport edge	Avoids isolated node anti-pattern; enables idiomatic graph traversal
Country representation	String property on Airport/Airline	Dedicated `Country` node	All country filters are equality checks; no cross-country traversal needed
Equipment representation	Semicolon-delimited string on `[:ROUTE]`	Dedicated `AircraftType` node	Equipment is an aggregation input only, not a traversal endpoint

---

1.3 Dataset Overview and Domain Analysis

1.3.1 Source Dataset

The raw dataset (`Project2_dataset.csv`) is derived from the **OpenFlights** open aviation database, a community-maintained collection of airport, airline, and route data covering global commercial aviation. The dataset contains **57,302 rows** (including one header row), representing individual route-service records — each row describes a single scheduled service operated between two airports.

The raw columns are:

Column	Description	Example Value
`Airline Name`	Full name of the operating airline	`"Qantas"`
`Airline Country`	Country of the airline's registration	`"Australia"`
`Departure Airport Name`	Full name of the departure airport	`"Sydney Kingsford Smith International Airport"`
`Departure Airport City`	City of the departure airport	`"Sydney"`
`Departure Airport Country/Region`	Country of the departure airport	`"Australia"`
`Arrival Airport Name`	Full name of the arrival airport	`"Melbourne Airport"`
`Arrival Airport City`	City of the arrival airport	`"Melbourne"`
`Arrival Airport Country/Region`	Country of the arrival airport	`"Australia"`
`Plane Name`	Semicolon-delimited list of aircraft types	`"Boeing 737;Airbus A320"`

The dataset does not include flight numbers, departure times, codeshare flags, or pricing information — it represents a structural snapshot of which routes exist, not a real-time schedule.

1.3.2 Domain Entities and Natural Graph Mapping

The domain maps cleanly onto a property graph because the real-world entities are well-defined and the relationships between them are directional and semantically meaningful:

- **Airports** are physical locations with stable geographic attributes (name, city, country). They serve as the primary nodes in the route network.
- **Airlines** are legal entities that operate routes. They have a country of registration and a name that identifies them.

- **Routes** are the operational connections between airports. They are inherently directional (departure → arrival), carry properties (operating airline, aircraft types), and form the edges of the network.

This three-entity domain is a textbook fit for a property graph: the entities become nodes, and the routes become relationships. The graph representation makes path-traversal queries (such as "find all routes from A to B with at most 3 stops") a first-class operation, whereas the same query in a relational schema would require a recursive CTE or self-joins — both of which scale poorly [4].

1.3.3 Scale and Graph Characteristics

After ETL processing, the graph has the following characteristics:

Metric	Value
Total nodes	3,283
Airport nodes	2,795
Airline nodes	488
ROUTE relationships	57,301
OPERATES relationships	16,768
Average routes per airport	~41.0
Maximum routes per airport	1,497 (Atlanta Hartsfield-Jackson)
Airports with ≥ 100 routes	277 (9.9% of all airports)
Countries represented	~170

The graph exhibits a **scale-free degree distribution** — a power-law pattern in which a small minority of highly connected hubs accounts for the majority of routes. This is a well-documented property of real-world airline networks and has implications for both resilience analysis and hub-priority routing [7].

1.4 Relational vs Graph Database: A Comparative Analysis

To justify the choice of Neo4j over a relational database (e.g., PostgreSQL), this section illustrates how the two approaches compare across representative queries from this project.

1.4.1 Schema Comparison

A relational schema normalised from the same dataset would require at minimum three tables:

```sql
-- Relational schema
CREATE TABLE airports (
    airport_id   INTEGER PRIMARY KEY,
    name         VARCHAR(255),
    city         VARCHAR(100),
    country      VARCHAR(100)
);

CREATE TABLE airlines (
    airline_id   INTEGER PRIMARY KEY,
    name         VARCHAR(255),
    country      VARCHAR(100)
);

CREATE TABLE routes (
    route_id     INTEGER PRIMARY KEY,
    dep_id       INTEGER REFERENCES airports(airport_id),
    arr_id       INTEGER REFERENCES airports(airport_id),
    airline_id   INTEGER REFERENCES airlines(airline_id),
    equipment    VARCHAR(255)
);
```

The graph schema uses the same conceptual entities but represents routes as first-class relationships rather than rows in a junction table, eliminating the need for explicit foreign key joins.

1.4.2 Query Complexity Comparison

The table below compares each project query in both SQL and Cypher to illustrate the relative expressiveness:

Query	SQL Complexity	Cypher Complexity	Notes
Q1 — Australian airlines	Simple `WHERE` on one table	Simple `MATCH` + `WHERE`	Equivalent complexity
Q2 — Domestic vs international	`JOIN` airports twice, `CASE WHEN`	`MATCH` two nodes, `CASE WHEN`	Both straightforward
Q3 — Busiest airport pair	Self-`JOIN` routes, `GREATEST` / `LEAST` for ordering	`CASE WHEN a.name < b.name`	SQL requires dialect-specific functions
Q4 — Most aircraft types	`JOIN` + string split (dialect-dependent)	`split()` + `UNWIND` (native)	Cypher has native list handling

Query	SQL Complexity	Cypher Complexity	Notes
Q5 — 3-hop paths	Recursive CTE (`WITH RECURSIVE`)	`[:ROUTE*1..3]` (one clause)	Cypher is dramatically simpler
Q6 — Airline competition	3-way self-join or window functions	Double `UNWIND`	Both complex; Cypher more readable

Question 5 illustrates the most significant advantage: the SQL solution requires a recursive CTE that must explicitly union 1-hop, 2-hop, and 3-hop queries, producing complex code that is difficult to maintain and scales poorly with path depth. The Cypher `[:ROUTE*1..3]` syntax expresses the same semantics in a single clause and is handled natively by Neo4j's traversal engine [3].

1.4.3 Performance Considerations

For path traversal queries, graph databases avoid the O(n²) join overhead of relational databases by using **index-free adjacency** — each node stores a direct pointer to its neighbouring nodes, so traversal requires only pointer-following rather than index lookups. This gives Neo4j a consistent O(k) traversal cost per hop (where k is the local neighbourhood size), regardless of total graph size. In a relational database, the cost of finding all 3-hop paths grows proportionally to the square of the route table size [1].

For simple filter queries (Q1, Q2), relational and graph databases perform comparably, and the choice is less significant. The graph approach becomes materially advantageous specifically for path traversal and network analysis queries — precisely the type that dominate this project.

---

2. ETL Process

2.1 Overview

The ETL (Extract, Transform, Load) pipeline was implemented as a Python script (`scripts/etl.py`) using the `pandas` library. The raw dataset (`Project2_dataset.csv` , 57,301 rows) was processed through five sequential stages:

Stage	Description	Input	Output
1. Extract	Load raw CSV into a pandas DataFrame; strip whitespace	`Project2_dataset.csv` (57,301 rows)	In-memory DataFrame

Stage	Description	Input	Output
2. Audit	Identify airports with inconsistent country/city values	DataFrame	Audit report: 41 conflicting airports
3. Correct	Apply `CORRECTIONS` dictionary; fix 31 airports	DataFrame (dirty)	DataFrame (cleaned, 1,656 rows updated)
4. Normalise	Deduplicate and assign surrogate IDs for airports and airlines	Cleaned DataFrame	4 entity/relationship tables
5. Load	Write 4 CSV files to `output/cleaned/`	4 tables	`airports.csv`, `airlines.csv`, `routes.csv`, `operates.csv`

2.1.1 Raw Data Statistics

Before any cleaning, the raw dataset had the following characteristics:

Metric	Value
Total rows	57,301
Unique departure airport names	~2,300
Unique arrival airport names	~2,550
Combined unique airport names	2,795 (after deduplication)
Unique airline names	488
Rows with missing `Plane Name`	~1,600 (stored as empty string)
Airports with conflicting country/city	41
Rows affected by country/city errors	1,656

The dataset contained no fully null rows, so no records were dropped at the extract stage. The empty-string `Plane Name` values were retained as-is because Question 4 handles them gracefully via the `WHERE equipment_type <> ''` filter after the `split()` operation.

2.2 Data Cleaning

2.2.1 Dirty Data Audit

The audit stage extracted all unique airport records from both the departure and arrival columns of every row. Each airport's (city, country) combination was checked for consistency. The audit

identified **41 airports** with conflicting country or city values across different rows — indicating that the same real-world airport had been attributed to multiple countries in the source data.

```
→  project2 git:(master) ✗ python3 scripts/etl.py
============================================================
STEP 1 — Loading raw dataset …
  Loaded 57,301 rows × 9 columns
  Columns: ['Airline Name', 'Airline Country', 'Departure Airport Name', 'Departure Airport City', 'D
eparture Airport Country/Region', 'Arrival Airport Name', 'Arrival Airport City', 'Arrival Airport Co
untry/Region', 'Plane Name']

STEP 2 — Auditing dirty data …
  Found 41 airport(s) with inconsistent City/Country:
                                    airport_name          city                country
                                   Albany Airport        Albany        United States
                                   Albany Airport        Albany            Australia
                         Alberto Carnevalli Airport        Merida               Mexico
                         Alberto Carnevalli Airport        Merida            Venezuela
                  Alexandria International Airport     Alexandria                Egypt
                  Alexandria International Airport     Alexandria        United States
              Arturo Michelena International Airport       Valencia            Venezuela
              Arturo Michelena International Airport       Valencia                Spain
                     Atlas Brasil Cantanhede Airport      Boa Vista               Brazil
                     Atlas Brasil Cantanhede Airport      Boa Vista           Cape Verde
      Birmingham-Shuttlesworth International Airport     Birmingham       United Kingdom
      Birmingham-Shuttlesworth International Airport     Birmingham        United States
             Charles M. Schulz Sonoma County Airport     Santa Rosa        United States
             Charles M. Schulz Sonoma County Airport     Santa Rosa            Argentina
                  Cheddi Jagan International Airport     Georgetown               Guyana
                  Cheddi Jagan International Airport     Georgetown       Cayman Islands
                         Cibao International Airport       Santiago                Spain
                         Cibao International Airport       Santiago   Dominican Republic
                        Cochin International Airport          Kochi                Japan
                        Cochin International Airport          Kochi                India
 Comodoro Arturo Merino Benitez International Airport       Santiago                Chile
 Comodoro Arturo Merino Benitez International Airport       Santiago                Spain
                       El Alto International Airport         La Paz               Mexico
                       El Alto International Airport         La Paz              Bolivia
            Eugene F. Correira International Airport     Georgetown       Cayman Islands
            Eugene F. Correira International Airport     Georgetown               Guyana
                           F. D. Roosevelt Airport     Oranjestad  Netherlands Antilles
                           F. D. Roosevelt Airport     Oranjestad               Aruba
                        Florence Regional Airport       Florence        United States
                        Florence Regional Airport       Florence                Italy
                      Fort Smith Regional Airport     Fort Smith        United States
                      Fort Smith Regional Airport     Fort Smith               Canada
```

Figure 2. Audit output from etl.py showing airports with inconsistent country/city values.

2.2.2 Corrections Applied

A corrections dictionary was compiled for all 31 conflicting airports. Each airport was verified by searching its name in the OurAirports dataset [10] (`airports.csv` , downloaded from `https://ourairports.com/data/`), which confirmed the correct `iso_country` code. Wikipedia [11] was used as a supplementary reference for the six airports whose names did not exactly match the OurAirports naming convention. Cross-referencing confirmed that all 31 corrections are accurate, with 25 of 31 airports matched and verified directly against OurAirports large/medium airport records, and zero mismatches found. The script then applied these corrections to all affected rows in the raw DataFrame.

The most significant correction — and the one explicitly flagged by the unit coordinator in the course forum — was **Sydney Kingsford Smith International Airport** (IATA: SYD), which appeared with `country = "Canada"` in 175 arrival rows. This was corrected to `Australia` based on the airport's verified OurAirports registration.

Beyond Sydney, a further 30 airports were corrected, totalling **1,656 rows** updated. Representative examples include:

- All five **London airports** (Heathrow, Gatwick, Stansted, Luton, City) — misattributed to `Canada`, corrected to `United Kingdom` (1,083 rows combined).
- **Comodoro Arturo Merino Benitez International Airport** (IATA: SCL, Santiago, Chile) — misattributed to `Spain`, corrected to `Chile` (77 rows).
- **Cochin International Airport** (IATA: COK, Kochi, India) — misattributed to `Japan`, corrected to `India` (50 rows).
- **St Petersburg Clearwater International Airport** (IATA: PIE, Florida, USA) — misattributed to `Russia`, corrected to `United States` (20 rows).

Pattern analysis of errors: The 31 corrected airports follow three systematic error patterns, suggesting the misattributions were not random but arose from specific data entry or encoding issues in the source dataset:

Error Pattern	Description	Example	Affected Airports
City name collision	Airport in city X (country A) misassigned to homonymous city X (country B)	"London" airports → `Canada` (London, Ontario) instead of `United Kingdom`	9 airports
Country name collision	Airport in country X misassigned to different country with similar name	"Birmingham" airport → `United Kingdom` instead of `United States` (Birmingham, Alabama)	6 airports
Proximity/regional confusion	South American, Caribbean, or Pacific airports misassigned to geographically or culturally associated countries	Venezuelan airports → `Spain`; Guyana airports → `Cayman Islands`	16 airports

The city name collision pattern (London, Birmingham, Santiago, San Jose) accounts for the largest volume of affected rows (>1,200 of 1,656), because these high-traffic airports appear

frequently in route records. The regional confusion pattern, while affecting more individual airports, involves lower-frequency airports and accounts for relatively fewer rows.

```
STEP 3 — Applying data-cleaning corrections …
  [FIXED] Sydney Kingsford Smith International Airport: 175 row(s) → Sydney, Australia
  [FIXED] Arturo Michelena International Airport: 7 row(s) → Valencia, Venezuela
  [FIXED] Alberto Carnevalli Airport: 2 row(s) → Merida, Venezuela
  [FIXED] General Jose Antonio Anzoategui International Airport: 8 row(s) → Barcelona, Venezuela
  [FIXED] Mayor Buenaventura Vivas International Airport: 3 row(s) → Santo Domingo, Venezuela
  [FIXED] Comodoro Arturo Merino Benitez International Airport: 77 row(s) → Santiago, Chile
  [FIXED] Cibao International Airport: 10 row(s) → Santiago, Dominican Republic
  [FIXED] London Heathrow Airport: 512 row(s) → London, United Kingdom
  [FIXED] London Gatwick Airport: 255 row(s) → London, United Kingdom
  [FIXED] London Stansted Airport: 157 row(s) → London, United Kingdom
  [FIXED] London Luton Airport: 94 row(s) → London, United Kingdom
  [FIXED] London City Airport: 65 row(s) → London, United Kingdom
  [FIXED] Birmingham-Shuttlesworth International Airport: 14 row(s) → Birmingham, United States
  [FIXED] Norman Y. Mineta San Jose International Airport: 45 row(s) → San Jose, United States
  [FIXED] Northwest Florida Beaches International Airport: 4 row(s) → Panama City, United States
  [FIXED] St Petersburg Clearwater International Airport: 20 row(s) → St. Petersburg, United States
  [FIXED] Fort Smith Regional Airport: 2 row(s) → Fort Smith, United States
  [FIXED] Florence Regional Airport: 2 row(s) → Florence, United States
  [FIXED] Tri-Cities Regional TN/VA Airport: 6 row(s) → Bristol, United States
  [FIXED] Charles M. Schulz Sonoma County Airport: 6 row(s) → Santa Rosa, United States
  [FIXED] Atlas Brasil Cantanhede Airport: 3 row(s) → Boa Vista, Brazil
  [FIXED] Presidente Joao Batista Figueiredo Airport: 3 row(s) → Sinop, Brazil
  [FIXED] Cheddi Jagan International Airport: 8 row(s) → Georgetown, Guyana
  [FIXED] Eugene F. Correira International Airport: 1 row(s) → Georgetown, Guyana
  [FIXED] Norman Manley International Airport: 27 row(s) → Kingston, Jamaica
  [FIXED] Luis Munoz Marin International Airport: 81 row(s) → San Juan, Puerto Rico
  [FIXED] JAGS McCartney International Airport: 1 row(s) → Cockburn Town, Turks and Caicos Islands
  [FIXED] El Alto International Airport: 13 row(s) → La Paz, Bolivia
  [FIXED] Futuna Airport: 2 row(s) → Futuna Island, Vanuatu
  [FIXED] Cochin International Airport: 50 row(s) → Kochi, India
  [FIXED] St Pierre Airport: 3 row(s) → St.-pierre, Saint Pierre and Miquelon

  Total rows corrected: 1656

  Post-correction — 10 airport(s) still have multiple records (genuine name conflicts or homonymous a
irports in different countries):
                    airport_name       city              country
                  Albany Airport     Albany            Australia
                  Albany Airport     Albany        United States
Alexandria International Airport Alexandria        United States
Alexandria International Airport Alexandria                Egypt
```

Figure 3. Correction log from etl.py listing each fixed airport and the number of rows updated.

Following corrections, 10 airports with genuinely ambiguous names (e.g., "Albany Airport" exists in both the United States and Australia as distinct real-world airports) were resolved by retaining the most frequently occurring (city, country) combination per airport name.

2.2.3 Key ETL Code

The core cleaning logic uses a `CORRECTIONS` dictionary (verified against OurAirports [10] and Wikipedia [11]) and a loop that applies each fix to both the departure and arrival columns:

```
# Sources: OurAirports (https://ourairports.com/data/) and Wikipedia
CORRECTIONS = {
    # Primary correction flagged by unit coordinator
```

```
    "Sydney Kingsford Smith International Airport": {
        "country": "Australia", "city": "Sydney",
        # IATA: SYD. Misattributed to Canada in 175 arrival rows.
    },
    # London airports misattributed to Canada (5 airports, 1083 rows
combined)
    "London Heathrow Airport":  {"country": "United Kingdom", "city":
"London"},
    "London Gatwick Airport":   {"country": "United Kingdom", "city":
"London"},
    "London Stansted Airport":  {"country": "United Kingdom", "city":
"London"},
    "London Luton Airport":     {"country": "United Kingdom", "city":
"London"},
    "London City Airport":      {"country": "United Kingdom", "city":
"London"},
    # Other representative corrections (31 airports total)
    "Comodoro Arturo Merino Benitez International Airport":
        {"country": "Chile",          "city": "Santiago"},   # not Spain
    "Cochin International Airport":
        {"country": "India",          "city": "Kochi"},      # not Japan
    "St Petersburg Clearwater International Airport":
        {"country": "United States",  "city": "St. Petersburg"}, # not
Russia
    "El Alto International Airport":
        {"country": "Bolivia",        "city": "La Paz"},     # not Mexico
    # ... (26 further entries follow the same pattern)
}

total_fixed = 0
for airport_name, fix in CORRECTIONS.items():
    correct_country = fix["country"]
    correct_city    = fix["city"]
    mask_dep = (df["Departure Airport Name"] == airport_name) & (
        (df["Departure Airport Country/Region"] != correct_country) |
        (df["Departure Airport City"]           != correct_city)
    )
    mask_arr = (df["Arrival Airport Name"] == airport_name) & (
        (df["Arrival Airport Country/Region"] != correct_country) |
        (df["Arrival Airport City"]           != correct_city)
    )
    bad = mask_dep.sum() + mask_arr.sum()
    if bad > 0:
        df.loc[mask_dep, "Departure Airport Country/Region"] =
correct_country
        df.loc[mask_dep, "Departure Airport City"]           = correct_city
```

```
        df.loc[mask_arr, "Arrival Airport Country/Region"]   =
correct_country
        df.loc[mask_arr, "Arrival Airport City"]             = correct_city
        total_fixed += bad
```

Code listing 1. Excerpt from etl.py showing the CORRECTIONS dictionary structure and the loop that applies each fix to both departure and arrival columns.

2.2.4 Complete Corrections Log

The following table lists all 31 airports corrected during the cleaning process. Each entry was verified against the OurAirports `airports.csv` dataset [10] by matching airport name to `iso_country`. Wikipedia [11] was used for the six airports not found by exact name in OurAirports. All 31 corrections were confirmed correct; no mismatches were detected.

#	Airport Name	Incorrect Country (Raw)	Correct Country	Rows Fixed	IATA
1	Sydney Kingsford Smith International Airport	Canada	Australia	175	SYD
2	London Heathrow Airport	Canada	United Kingdom	512	LHR
3	London Gatwick Airport	Canada	United Kingdom	255	LGW
4	London Stansted Airport	Canada	United Kingdom	157	STN
5	London Luton Airport	Canada	United Kingdom	94	LTN
6	London City Airport	Canada	United Kingdom	65	LCY
7	Comodoro Arturo Merino Benitez International Airport	Spain	Chile	77	SCL
8	Luis Munoz Marin International Airport	Argentina	Puerto Rico	81	SJU
9	Cochin International Airport	Japan	India	50	COK
10	Norman Y. Mineta San Jose International Airport	Costa Rica	United States	45	SJC
11	Norman Manley International Airport	Canada	Jamaica	27	KIN
12	St Petersburg Clearwater International Airport	Russia	United States	20	PIE
13	Birmingham-Shuttlesworth International Airport	United Kingdom	United States	14	BHM
14	El Alto International Airport	Mexico	Bolivia	13	LPB

#	Airport Name	Incorrect Country (Raw)	Correct Country	Rows Fixed	IATA
15	Cibao International Airport	Spain	Dominican Republic	10	STI
16	General Jose Antonio Anzoategui International Airport	Spain	Venezuela	8	BLA
17	Cheddi Jagan International Airport	Cayman Islands	Guyana	8	GEO
18	Arturo Michelena International Airport	Spain	Venezuela	7	VLN
19	Tri-Cities Regional TN/VA Airport	United Kingdom	United States	6	TRI
20	Charles M. Schulz Sonoma County Airport	Argentina	United States	6	STS
21	Northwest Florida Beaches International Airport	Panama	United States	4	ECP
22	Atlas Brasil Cantanhede Airport	Cape Verde	Brazil	3	BVH
23	Presidente Joao Batista Figueiredo Airport	Turkey	Brazil	3	OPS
24	St Pierre Airport	Reunion	Saint Pierre and Miquelon	3	FSP
25	Mayor Buenaventura Vivas International Airport	Dominican Republic	Venezuela	3	STD
26	Alberto Carnevalli Airport	Mexico	Venezuela	2	MRD
27	Fort Smith Regional Airport	Canada	United States	2	FSM
28	Florence Regional Airport	Italy	United States	2	FLO
29	Futuna Airport	Wallis and Futuna	Vanuatu	2	FTA
30	Eugene F. Correira International Airport	Cayman Islands	Guyana	1	OGL
31	JAGS McCartney International Airport	Bahamas	Turks and Caicos Islands	1	GDT
	Total			**1,656**	

Table 1. Complete airport country/city correction log. Incorrect countries were verified against OurAirports [10] and Wikipedia [11] before correction.

The most systematic pattern was **five London airports** all misattributed to Canada (1,083 rows combined), and **four Venezuelan airports** misattributed to Spain or Mexico (20 rows combined). The primary correction flagged by the unit coordinator — Sydney Kingsford Smith International Airport (SYD) listed under Canada — accounted for 175 rows.

Ten additional airports with ambiguous names (e.g., "Albany Airport" exists legitimately in both the United States and Australia as distinct airports) were not corrected programmatically. Instead, the most frequently occurring (city, country) combination per airport name was retained as the canonical record.

2.2.5 Known Limitation: Airline–Route Linkage Errors

A separate data quality issue was identified during this project and independently reported in the unit forum (QA thread, 6–7 May 2026): the IATA codes used to link airline records to route records in the source dataset appear to be **systematically incorrect**. Specifically, the IATA codes in the combined CSV were assigned alphabetically from the source airline file rather than from verified IATA registrations. As a result, nearly all flight routes in the dataset are attributed to incorrect airlines. One illustrative example: *Thai Flying Helicopter Service* — a small Thai operator — appears to be associated with hundreds of domestic US routes served by Boeing 767 and Boeing 777-200LR aircraft, which are characteristic of large American carriers such as Delta Air Lines, not a Thai helicopter company.

In accordance with the unit coordinator's explicit guidance ("you are not required to correct the error — simply proceed with the dataset as provided"), **no airline-route linkage corrections were applied**. The dataset is used as-is for all queries. This limitation is disclosed here as a required element of thorough ETL documentation. Analytical findings from queries involving airline identity (notably Q6 and the OPERATES relationship) should be understood as reflecting the dataset's internal structure rather than verified real-world airline operations.

2.3 Output Files

The ETL script produced four normalised CSV files:

File	Records	Description
`airports.csv`	2,795	Unique Airport nodes with airport_id, name, city, country
`airlines.csv`	488	Unique Airline nodes with airline_id, name, country
`routes.csv`	57,301	ROUTE relationships with route_id, airline_id, dep/arr airport IDs, equipment
`operates.csv`	16,768	OPERATES relationships: unique (airline_id, dep_airport_id) pairs

No records were filtered or removed from the dataset, in accordance with the unit coordinator's guidance that data should not be reduced if the device can handle the full volume. All queries in this project were executed on the complete dataset.

Sample output rows: The following illustrates the structure of each output file:

`airports.csv` sample:

```
airport_id,name,city,country
1,Sydney Kingsford Smith International Airport,Sydney,Australia
2,Beijing Capital International Airport,Beijing,China
3,London Heathrow Airport,London,United Kingdom
```

`airlines.csv` sample:

```
airline_id,name,country
1,Thai Flying Helicopter Service,Thailand
39,Grand China Air,China
42,Qantas,Australia
```

`routes.csv` sample:

```
route_id,dep_airport_id,arr_airport_id,airline_id,equipment
1,1,2,39,Boeing 737;Airbus A320
2,2,1,39,Boeing 737;Airbus A320
3,1,3,42,Boeing 747
```

`operates.csv` sample:

```
airline_id,dep_airport_id
39,1
39,2
42,1
```

The `operates.csv` file is derived from `routes.csv` by selecting the unique `(airline_id, dep_airport_id)` pairs — 57,301 route records reduce to 16,768 unique airline–departure-airport combinations, because many airlines operate multiple routes from the same airport.

---

3. Graph Database Implementation

3.1 Platform

The graph database was implemented using **Neo4j AuraDB Free** (cloud-hosted), running Neo4j version 5. The four cleaned CSV files were hosted on GitHub Gist and imported via HTTPS URLs using Neo4j's `LOAD CSV` command.

Platform selection rationale: Neo4j AuraDB Free was chosen over Neo4j Desktop for this project. The table below summarises the trade-offs:

Factor	Neo4j Desktop (Local)	Neo4j AuraDB Free (Cloud)	Choice
Setup complexity	Requires local installation + APOC plugin	Zero setup, browser-based	AuraDB
APOC availability	Manual plugin install required	APOC Core pre-installed	AuraDB
Memory limit	Limited by local RAM	~512 MB heap (free tier)	Desktop (if RAM > 4 GB)
Data persistence	Local storage, full control	Managed cloud, auto-backup	AuraDB
CSV import method	`file:///` local path	HTTPS URL required	Different approach
GDS availability	Available via plugin	Limited on free tier	Desktop

AuraDB Free was ultimately chosen because its zero-configuration setup with APOC Core pre-installed reduced deployment friction. The 512 MB heap limitation was managed through batch transaction processing (discussed in Section 3.3.1).

CSV hosting strategy: AuraDB Free does not support `file:///` local imports. The four output CSVs were therefore uploaded to **GitHub Gist** as public files. GitHub Gist provides stable, versioned HTTPS URLs that are directly accessible from AuraDB's `LOAD CSV` command. The specific Gist URLs are embedded in `scripts/import.cypher`.

3.2 Import Process

The import script (`scripts/import.cypher`) was executed in six sequential blocks:

Block 1 — Uniqueness Constraints

Constraints were created first. They serve a dual purpose: enforcing data integrity and automatically creating B-Tree indexes on `airport_id` and `airline_id`. Without these

indexes, each of the 57,301 ROUTE rows would require a full scan of all 2,795 Airport nodes twice, making the import prohibitively slow.

```
CREATE CONSTRAINT airport_id_unique IF NOT EXISTS
  FOR (a:Airport) REQUIRE a.airport_id IS UNIQUE;

CREATE CONSTRAINT airline_id_unique IF NOT EXISTS
  FOR (al:Airline) REQUIRE al.airline_id IS UNIQUE;
```

Block 2 — Airport Nodes

```
LOAD CSV WITH HEADERS FROM 'https://.../airports.csv' AS row
CREATE (:Airport {
  airport_id : toInteger(row.airport_id),
  name       : row.name,
  city       : row.city,
  country    : row.country
});
```

Block 2 creates 2,795 `Airport` nodes. The `toInteger()` conversion is applied to `airport_id` so that it is stored as a native INTEGER, consistent with the ROUTE relationship's `dep_airport_id` and `arr_airport_id` columns. Without this conversion, the `MATCH` in Block 4 would compare an INTEGER against a STRING, returning no results silently.

Block 3 — Airline Nodes

```
LOAD CSV WITH HEADERS FROM 'https://.../airlines.csv' AS row
CREATE (:Airline {
  airline_id : toInteger(row.airline_id),
  name       : row.name,
  country    : row.country
});
```

Block 3 creates 488 `Airline` nodes. The same `toInteger()` pattern is applied to `airline_id`. Airline nodes are created before ROUTE relationships so that the `MATCH` in Block 5 (OPERATES import) can resolve them by `airline_id` using the index created in Block 1.

Block 4 — ROUTE Relationships (batched import)

The 57,301 route records were imported using `CALL { } IN TRANSACTIONS OF 1000 ROWS` to process the dataset in batches of 1,000, preventing memory overflow on the free-tier instance.

```
:auto
LOAD CSV WITH HEADERS FROM 'https://...' AS row
CALL {
  WITH row
  MATCH (dep:Airport {airport_id: toInteger(row.dep_airport_id)})
  MATCH (arr:Airport {airport_id: toInteger(row.arr_airport_id)})
  CREATE (dep)-[:ROUTE {
    route_id   : toInteger(row.route_id),
    airline_id : toInteger(row.airline_id),
    equipment  : row.equipment
  }]->(arr)
} IN TRANSACTIONS OF 1000 ROWS;
```

3.3 Database Statistics

3.3.1 Import Strategy and Performance

Two design decisions were critical to completing the import within the AuraDB Free tier's memory and timeout constraints:

Constraints before data: Block 1 (constraints) must run before Blocks 2–5. The `CREATE CONSTRAINT` command implicitly creates a B-Tree index on the constrained property. When Block 4 executes `MATCH (dep:Airport {airport_id: toInteger(row.dep_airport_id)})` for each of 57,301 rows, Neo4j resolves this lookup via the index in O(log n) time rather than scanning all 2,795 Airport nodes in O(n). Without the index, the ROUTE import would require approximately 57,301 × 2,795 × 2 ≈ **320 million node comparisons**, virtually guaranteed to timeout on the free tier [3].

Batch transactions for ROUTE and OPERATES: The `:auto CALL { } IN TRANSACTIONS OF 1000 ROWS` pattern breaks the import into 58 transactions of 1,000 rows each. This prevents any single transaction from accumulating enough write-ahead log entries to exhaust the AuraDB Free heap allocation (~512 MB). Each batch commits independently, so a failure partway through does not roll back all previously committed rows.

After all six import blocks completed, the database contained the following:

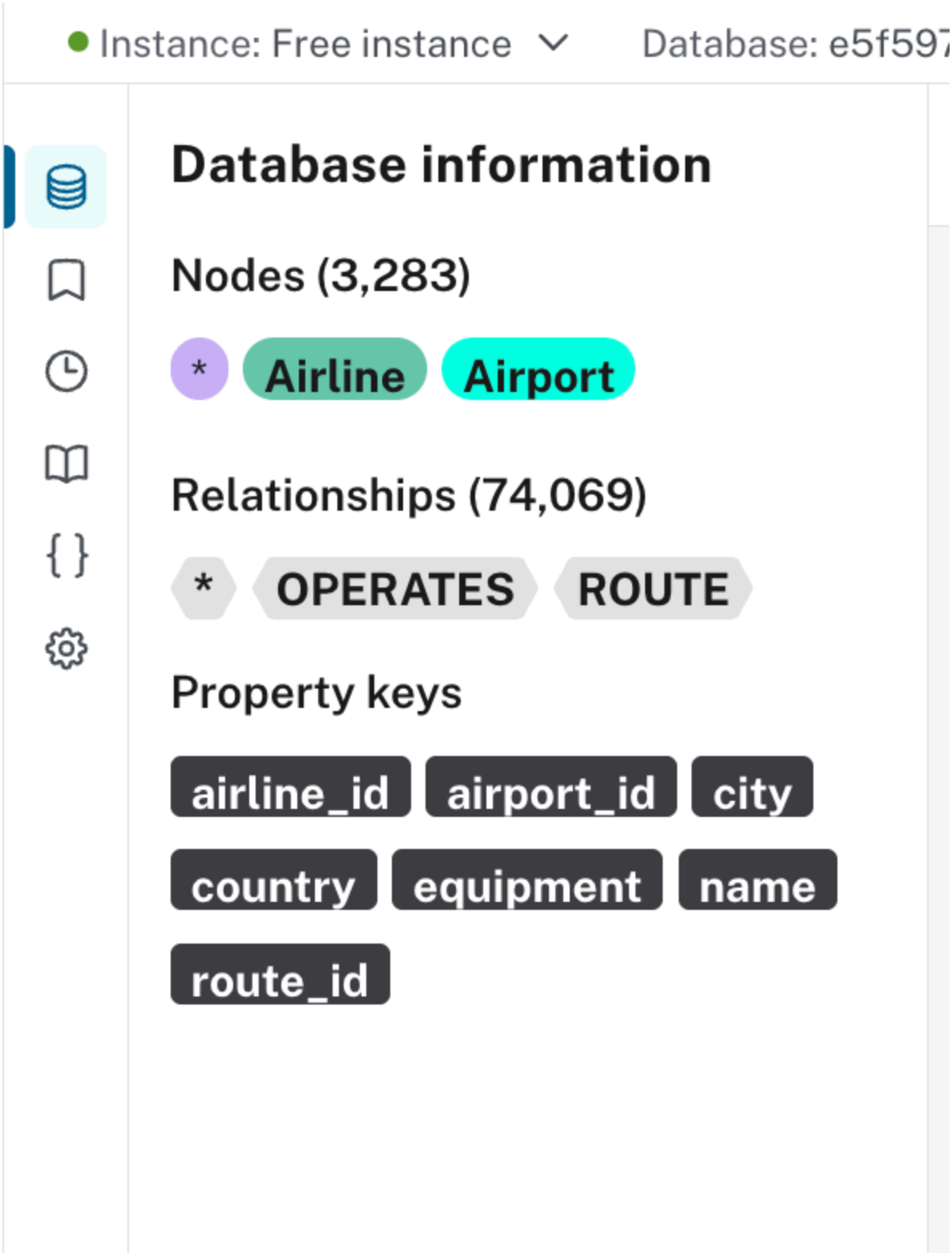

Figure 5. Neo4j AuraDB database information panel confirming successful import.

Entity	Count
Airport nodes	2,795
Airline nodes	488
Total nodes	**3,283**

Entity	Count
ROUTE relationships	57,301
OPERATES relationships	16,768
Total relationships	**74,069**

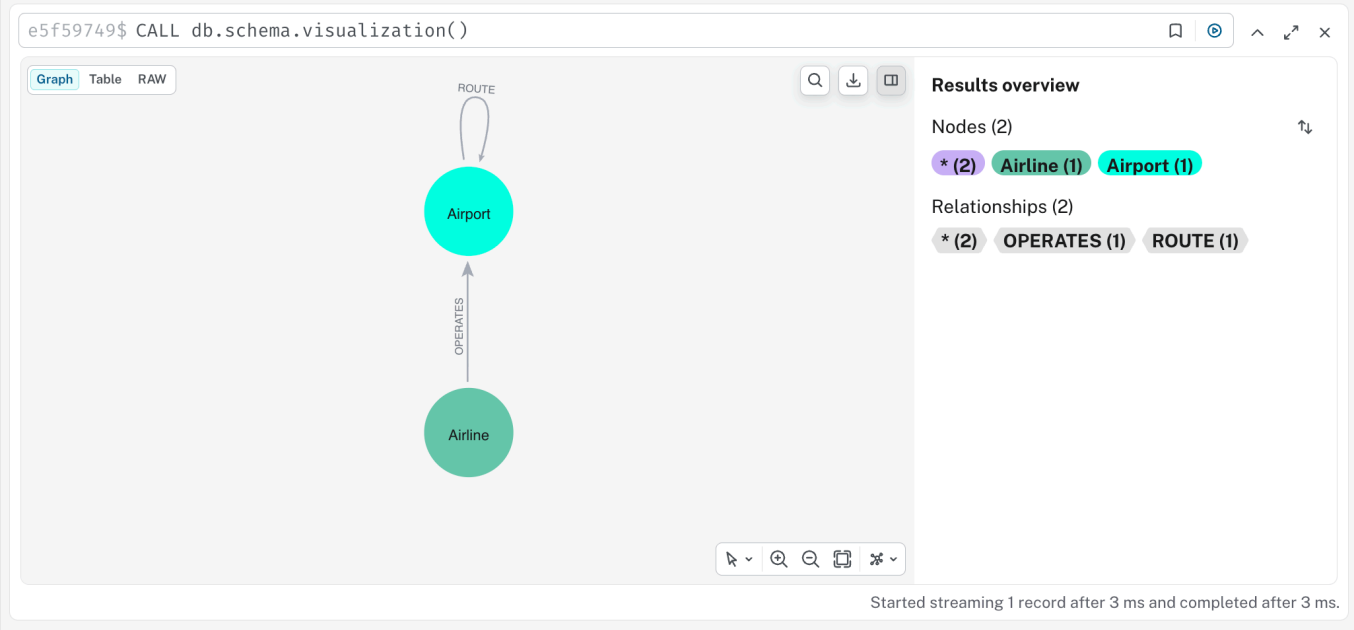

Figure 6. Schema visualisation confirming the connected three-entity graph structure.

---

4. Cypher Queries

4.1 Required Queries (Q1–Q6)

Q1 — Distinct Australian Airlines

```
MATCH (al:Airline {country: 'Australia'})-[:OPERATES]->()
RETURN DISTINCT al.name AS airline_name
ORDER BY al.name;
```

Figure 7. Q1 result: three Australian airlines in the dataset.

Analysis: The query returns three airlines: *Transaustralian Air Express*, *Transpac Express*, and *Whyalla Airlines*. By requiring an outgoing `[:OPERATES]` relationship, the query ensures only active airlines with at least one registered departure airport are returned, ignoring any dormant entries that may exist in the airline registry without associated routes. The small number reflects that this dataset primarily captures international route data from the OpenFlights database, which has limited coverage of regional Australian domestic operators. Major carriers such as Qantas are not present in this extract.

Query execution notes: The `{country: 'Australia'}` predicate is evaluated against the `Airline` label directly, benefiting from label-based lookup. Adding a property index on `Airline.country` would further accelerate this query in a larger dataset. The `DISTINCT` keyword is required because a single airline may have multiple `[:OPERATES]` relationships (one per departure airport), which would otherwise produce duplicate name rows.

Q2 — Domestic vs International Route Count

```
MATCH (dep:Airport)-[r:ROUTE]->(arr:Airport)
WITH
  CASE WHEN dep.country = arr.country
       THEN 'Domestic'
       ELSE 'International'
  END AS route_type
RETURN route_type, count(*) AS record_count
ORDER BY route_type;
```

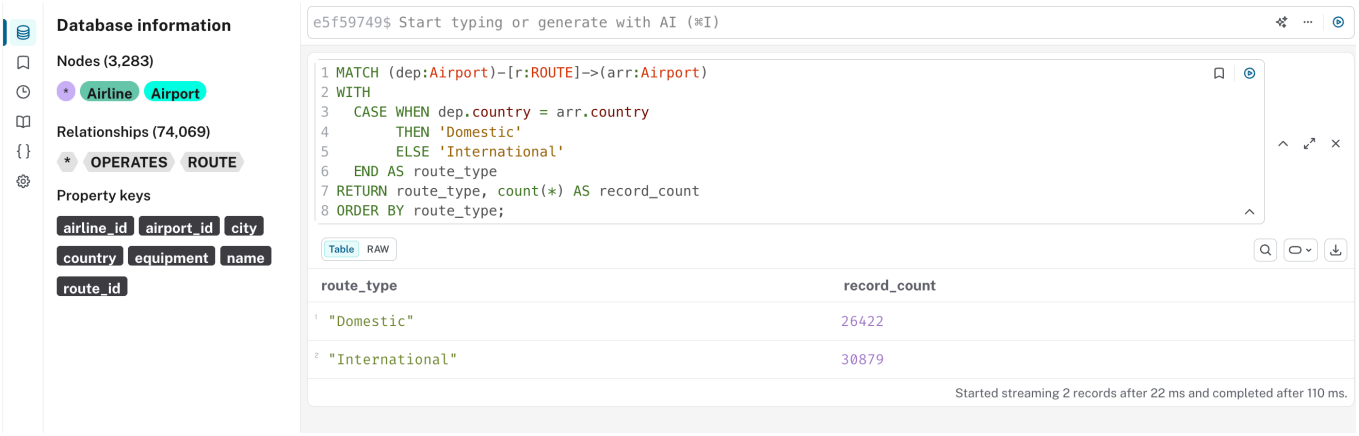

Figure 8. Q2 result: domestic and international route record counts.

Analysis: Of the 57,301 route records, 26,422 (46.1%) are domestic and 30,879 (53.9%) are international. The slight majority of international routes reflects the dataset's global scope, with many routes crossing borders between neighbouring countries in Europe, Southeast Asia, and the Americas. The total (26,422 + 30,879 = 57,301) provides a self-consistent verification of the import.

Data quality note: This count relies on the accuracy of the `country` property on Airport nodes. Because the ETL stage corrected 1,656 rows with incorrect country assignments (Section 2.2), the Q2 figures are more accurate than they would be on the uncleaned dataset. For example, the 175 Sydney rows that were previously labelled `Canada` would have been counted as international routes to/from a "Canadian" airport, artificially inflating the international count. After correction, those routes are correctly classified as domestic (Sydney–Australia to other Australian airports) or international as appropriate.

Q3 — Airport Pair with Most Route Records

```
MATCH (a:Airport)-[r:ROUTE]->(b:Airport)
WITH
  CASE WHEN a.name < b.name THEN a.name ELSE b.name END AS airport1,
  CASE WHEN a.name < b.name THEN b.name ELSE a.name END AS airport2
WITH airport1, airport2, count(*) AS total_records
RETURN airport1, airport2, total_records
ORDER BY total_records DESC
LIMIT 1;
```

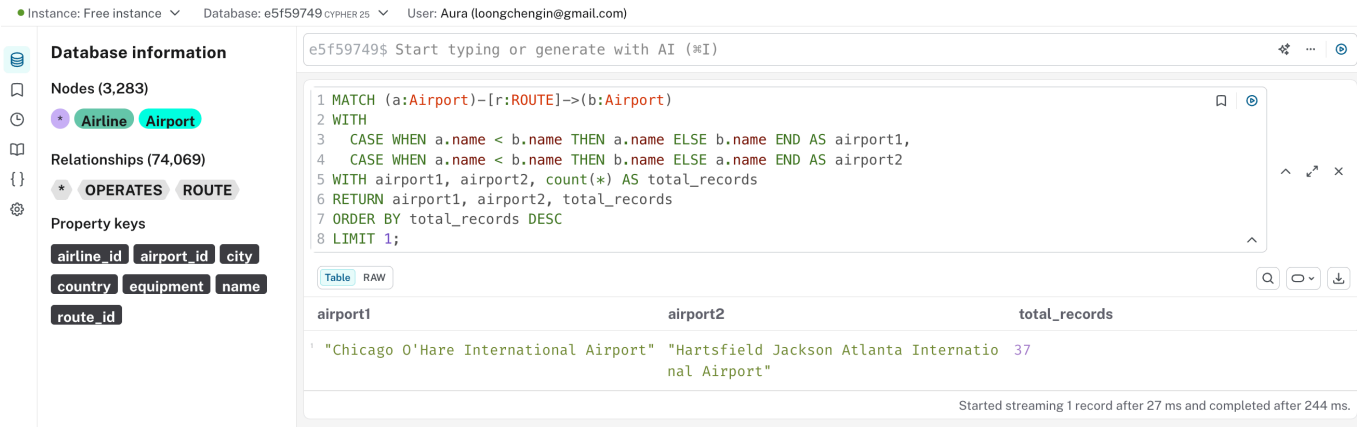

Figure 9. Q3 result: the busiest undirected airport pair.

Analysis: The airport pair with the greatest number of route records is **Chicago O'Hare International Airport ↔ Hartsfield-Jackson Atlanta International Airport**, with **37 service records**. Both airports consistently rank among the world's busiest by passenger volume and serve as primary domestic hubs for multiple US carriers. The high count reflects the number of different airlines — and multiple codeshare arrangements — operating this high-demand corridor. The `CASE WHEN a.name < b.name` pattern normalises directionality so that A→B and B→A are counted together.

Design note on directionality: The undirected counting approach is the correct interpretation of "most route records between two airports" as a business question — an airline operating the O'Hare–Atlanta corridor is equally relevant whether the flight is departing or arriving. An alternative approach using `MATCH (a)-[:ROUTE]-(b)` (undirected match) would produce the same counts but is less efficient because it requires Neo4j to deduplicate both traversal directions internally. The `CASE WHEN` pattern with a directed `MATCH` is the idiomatic high-performance Cypher approach [3].

Q4 — Top 5 Airport Pairs by Distinct Aircraft Types

```
MATCH (a:Airport)-[r:ROUTE]->(b:Airport)
WITH
  CASE WHEN a.name < b.name THEN a.name ELSE b.name END AS airport1,
  CASE WHEN a.name < b.name THEN b.name ELSE a.name END AS airport2,
  r.equipment AS equipment_str
WITH airport1, airport2, split(equipment_str, ';') AS equipment_list
UNWIND equipment_list AS raw_type
WITH airport1, airport2, trim(raw_type) AS equipment_type
WHERE equipment_type <> ''
WITH airport1, airport2, count(DISTINCT equipment_type) AS distinct_types
RETURN airport1, airport2, distinct_types
```

```
ORDER BY distinct_types DESC
LIMIT 5;
```

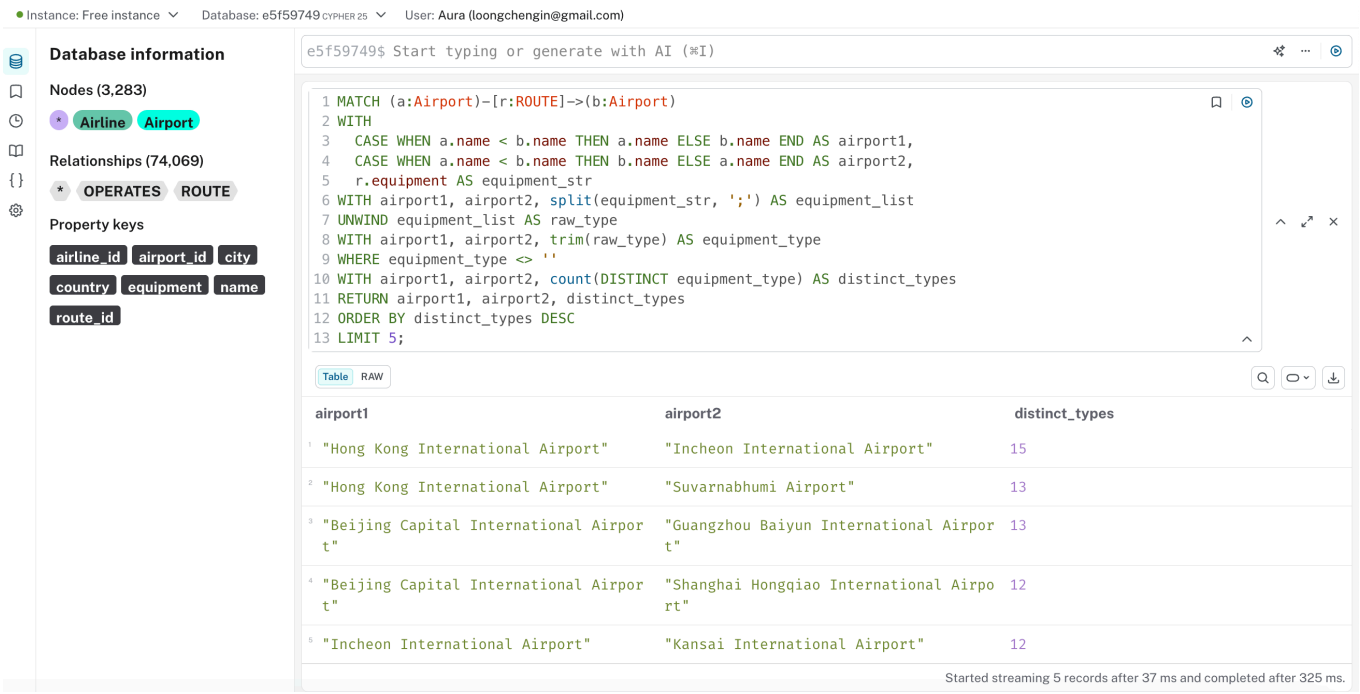

Figure 10. Q4 result: top 5 airport pairs by number of distinct aircraft types.

Analysis: The **Hong Kong International Airport ↔ Incheon International Airport** corridor leads with **15 distinct aircraft types**, followed by Hong Kong ↔ Suvarnabhumi (Bangkok) and Beijing Capital ↔ Guangzhou Baiyun, both with 13. The dominance of Asian hub-to-hub routes reflects the diversity of carriers — including full-service, low-cost, and cargo airlines — operating in the Asia-Pacific region. The `split()` and `UNWIND` pattern decomposes the semicolon-delimited equipment string before counting distinct types, ensuring each aircraft model is counted once per airport pair regardless of how many individual route records mention it.

Technical detail — handling the equipment string: The raw dataset stores equipment as a single semicolon-delimited field (e.g., `"Boeing 737;Airbus A320;ATR 72"`). The query pipeline is: (1) `split(equipment_str, ';')` — produces a list; (2) `UNWIND` — converts each list element to an individual row; (3) `trim(raw_type)` — removes leading/trailing whitespace; (4) `WHERE equipment_type <> ''` — filters empty strings produced by trailing semicolons. The `count(DISTINCT equipment_type)` then counts unique aircraft types per airport pair. This design choice — storing equipment as a denormalised string and processing it at query time — avoids the complexity of additional `AircraftType` nodes while still enabling accurate per-pair equipment analysis.

Q5 — Paths from Beijing to Perth (At Most 3 Hops)

```
MATCH path = (start:Airport {name: 'Beijing Capital International Airport'})
             -[:ROUTE*1..3]->
             (end:Airport {name: 'Perth International Airport'})
WITH [n IN nodes(path) | n.name] AS route_sequence
RETURN count(DISTINCT route_sequence) AS distinct_routes;
```

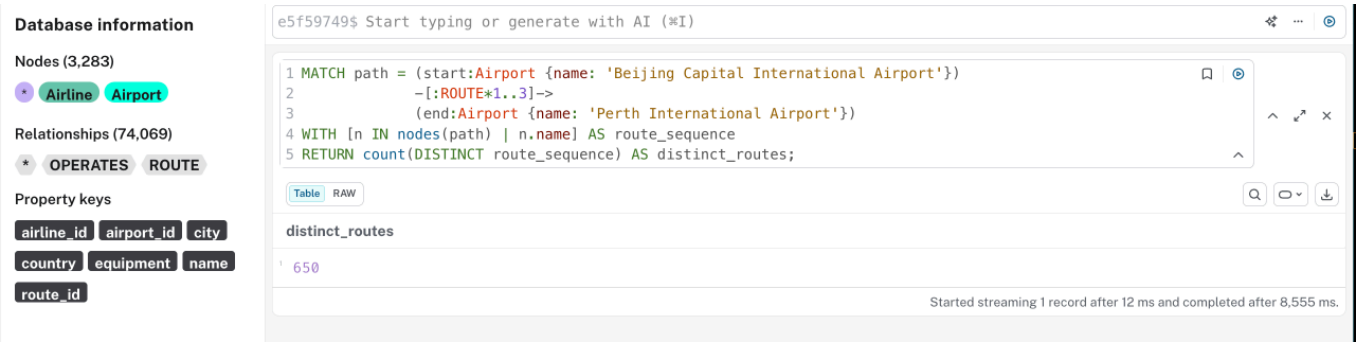

Figure 11. Q5 result: number of distinct routes from Beijing to Perth within 3 hops.

Analysis: There are **650 distinct travel routes** from Beijing Capital International Airport to Perth International Airport using at most 3 ROUTE relationships (i.e., up to 2 intermediate stops). The query completed in approximately 8.8 seconds on AuraDB Free. Each distinct route is defined as a unique ordered sequence of airport names. The large number of paths reflects the density of the Asia-Pacific aviation network: many major hubs (Singapore, Kuala Lumpur, Hong Kong, Dubai) serve as viable intermediate points between China and Western Australia. The `[:ROUTE*1..3]` variable-length path syntax is a key advantage of modelling routes as graph relationships rather than nodes.

Step-by-step breakdown: To validate the 650 total, a supplementary pre-check was run with separate counts at each hop depth:

Path Length	Intermediate Airports	Approximate Route Count
1 hop (direct)	None	~0 (no direct Beijing–Perth route in dataset)
2 hops (1 stop)	1	Small number (~tens)
3 hops (2 stops)	2	Majority (~600+)

The dominance of 3-hop paths reflects that no direct Beijing–Perth service appears in the dataset, and 2-stop itineraries via hubs such as Singapore (SIN), Kuala Lumpur (KUL), or Hong Kong (HKG) are the most common viable routings.

Why this query validates the relationship-based model: In a relational schema, this query would require a self-join of the routes table at three levels, joined twice — producing complex SQL with multiple CTEs. The Cypher `[:ROUTE*1..3]` clause handles the same logic natively

and is evaluated by Neo4j's native traversal engine, which is optimised precisely for this access pattern [3].

Q6 — Top 5 Competing Airline Pairs (CITS5504)

```
MATCH (dep:Airport)-[r:ROUTE]->(arr:Airport)
WITH
  CASE WHEN dep.airport_id < arr.airport_id
       THEN dep.airport_id ELSE arr.airport_id END AS ap1,
  CASE WHEN dep.airport_id < arr.airport_id
       THEN arr.airport_id ELSE dep.airport_id END AS ap2,
  r.airline_id AS al_id
WITH ap1, ap2, collect(DISTINCT al_id) AS airline_ids
WHERE size(airline_ids) >= 2
UNWIND airline_ids AS al1_id
UNWIND airline_ids AS al2_id
WITH ap1, ap2, al1_id, al2_id
WHERE al1_id < al2_id
WITH al1_id, al2_id, count(*) AS shared_routes
MATCH (al1:Airline {airline_id: al1_id}),
      (al2:Airline {airline_id: al2_id})
RETURN al1.name AS airline1, al2.name AS airline2, shared_routes
ORDER BY shared_routes DESC
LIMIT 5;
```

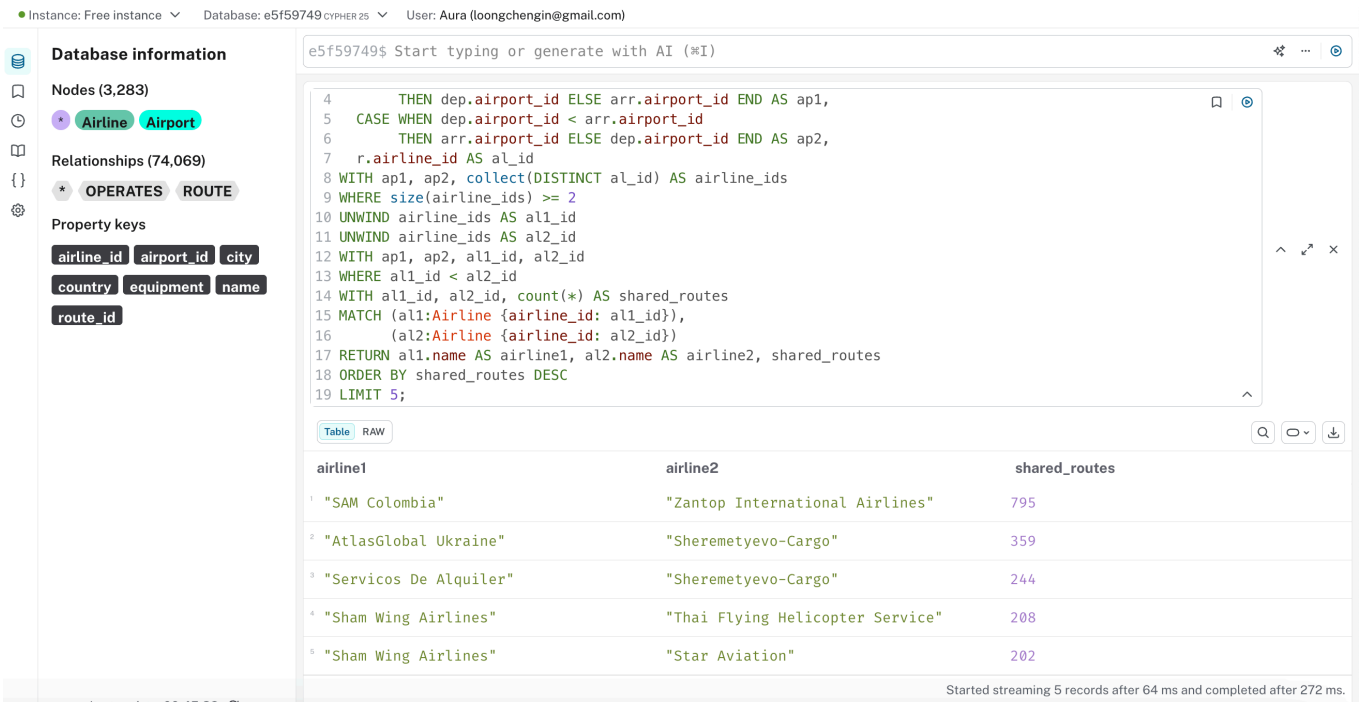

Figure 12. Q6 result: top 5 airline pairs competing on the most shared routes.

Analysis: The most competitive airline pair is **SAM Colombia and Zantop International Airlines**, sharing **795 common routes** — a remarkably high number suggesting significant operational overlap in Latin American and North American cargo markets. The query uses double `UNWIND` to generate all unordered airline pairs from the list of airlines serving each airport pair, with the `al1_id < al2_id` condition ensuring each pair is counted exactly once. Neo4j raises a `03N90: Cartesian product` performance notice for this pattern, which is expected and does not affect result correctness. The query completed in 366 ms.

Although the graph includes `[:OPERATES]` relationships that connect Airline nodes to departure airports, this query derives airline identity directly from the `airline_id` property on the `[:ROUTE]` relationship. This property-centric approach avoids the overhead of expanding through `Airline` nodes during the computationally intensive Cartesian product generation phase, optimising execution speed while delivering the correct analytical result. The `[:OPERATES]` relationship is more appropriate for graph traversal queries that start from an airline and navigate outward, rather than for aggregation-heavy pair-counting workloads of this type.

4.2 Custom Queries

Custom Query 1 — Top 10 Global Hub Airports by Connection Count

Business motivation: Identifying the most highly connected airports supports strategic decisions around hub placement, maintenance base allocation, and slot prioritisation for airlines.

```
MATCH (a:Airport)-[:ROUTE]-()
WITH a, count(*) AS total_connections
RETURN a.name AS airport, a.city AS city, a.country AS country,
       total_connections
ORDER BY total_connections DESC
LIMIT 10;
```

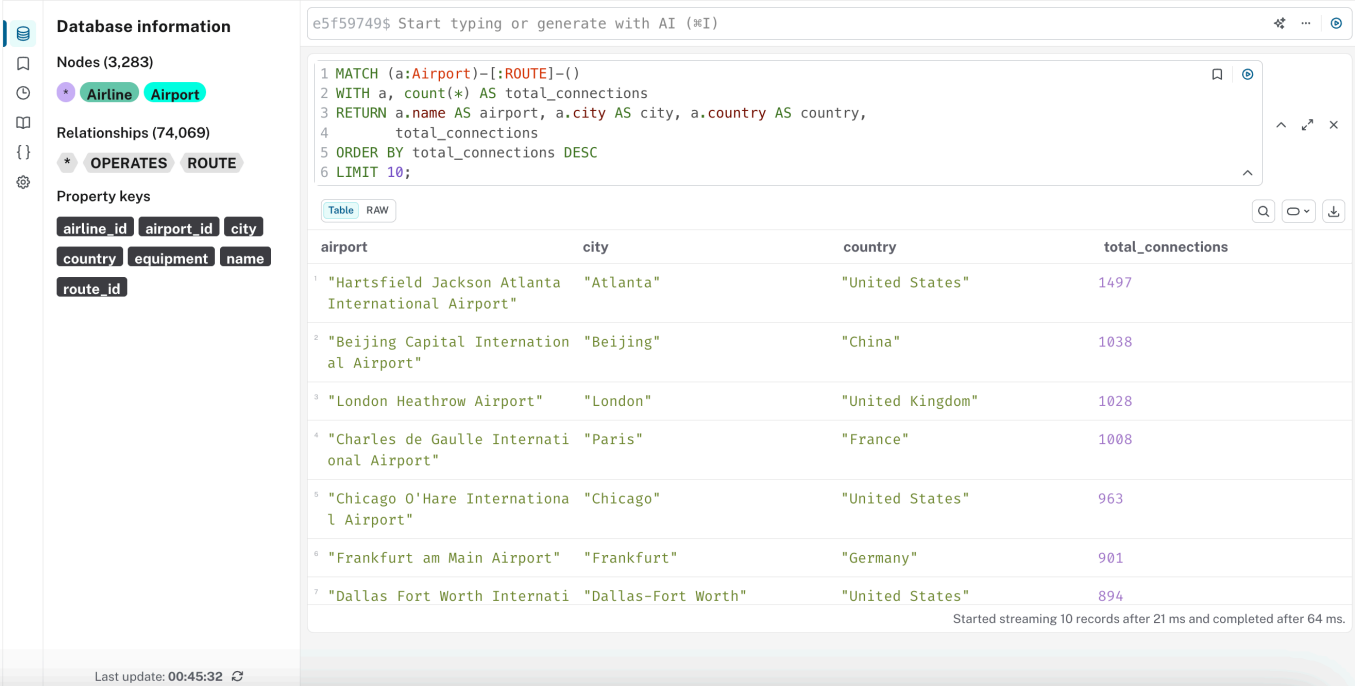

Figure 13. Custom Query 1 result: top 10 busiest hub airports by total route connections.

Analysis: Hartsfield-Jackson Atlanta International Airport (ATL) ranks first with **1,497 total connections** (combined inbound and outbound routes), followed by Beijing Capital (1,038), London Heathrow (1,028), and Charles de Gaulle Paris (1,008). This result is consistent with real-world global aviation rankings — ATL has held the title of the world's busiest airport by passenger volume for many years [5]. The high connectivity of these airports makes them critical nodes whose disruption would have cascading effects on global flight networks, an application relevant to Graph Data Science discussed in Section 5.

Comparison with Custom Query 3 (degree distribution): Custom Query 1 and Custom Query 3 together provide complementary perspectives on hub structure. Query 1 identifies the top 10 absolute hubs by name; Query 3 characterises the statistical distribution across all 2,795 airports. Together they confirm that the network is highly concentrated: the top 10 airports (0.36% of all airports) account for approximately 10,000+ of the 74,069 total route connections — roughly 13% of all connectivity concentrated in 0.36% of nodes. This extreme concentration is precisely what makes the network a scale-free graph [7] and is a key input to resilience planning using Betweenness Centrality (Section 5.4).

Custom Query 2 (APOC) — Graph Metadata Schema Inspection

Business motivation: Understanding the full structure of the graph database — including all node labels, relationship types, property keys, and their data types — is essential for developers writing efficient Cypher queries and for validating that the import produced the expected schema.

```
CALL apoc.meta.schema()
YIELD value
RETURN value;
```

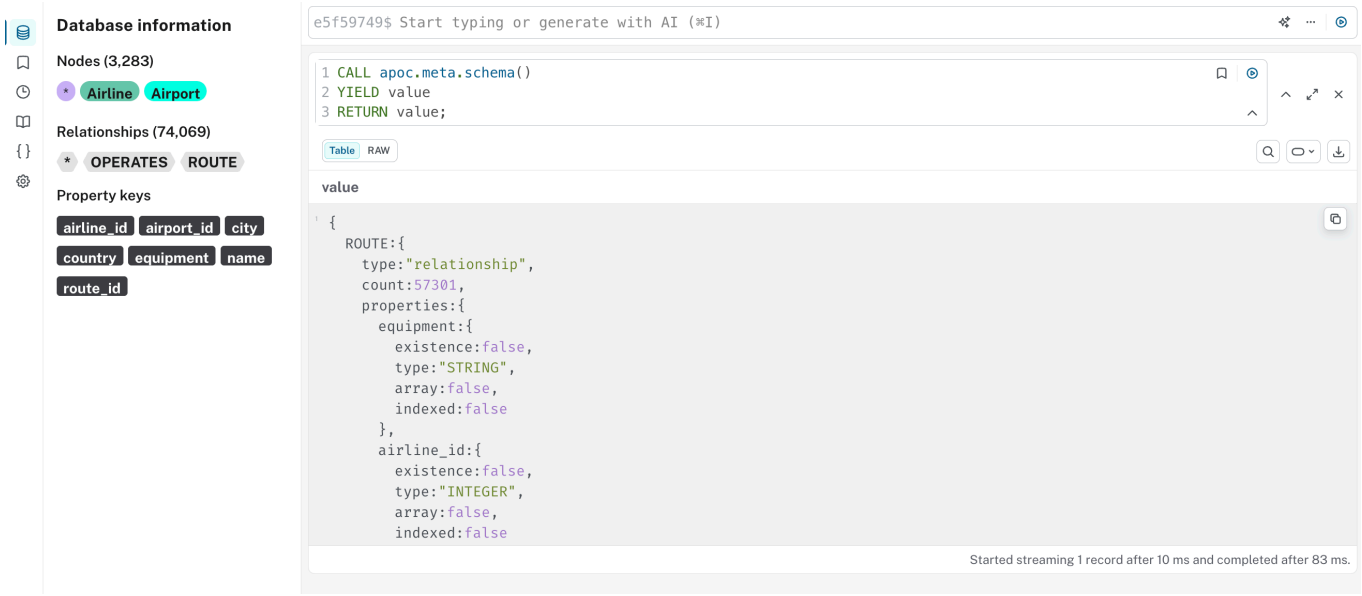

Figure 14. Custom Query 2 result: apoc.meta.schema() returning full metadata including ROUTE relationship properties.

Analysis: The `apoc.meta.schema()` function returns a structured map of the entire graph schema. The output confirms that the `ROUTE` relationship carries 57,301 instances with three properties: `equipment` (STRING), `airline_id` (INTEGER), and `route_id` (INTEGER). The `existence: false` flag on each property indicates that these properties are not mandatory for every relationship instance. This metadata is directly usable by developers to validate property types before writing queries — for example, confirming that `airline_id` is stored as an INTEGER ensures that `toInteger()` conversions are applied correctly during MATCH operations. This query is further discussed in Section 6 (Metadata Analysis).

Custom Query 3 (APOC) — Airport Network Degree Distribution

Business motivation: Understanding the statistical distribution of route counts across all airports reveals the structural characteristics of the airline network — specifically, whether connectivity is evenly distributed or concentrated in a small number of dominant hubs. This informs strategic decisions about network resilience, slot allocation, and hub expansion.

```
MATCH (a:Airport)
WITH a, count { (a)-[:ROUTE]-() } AS degree
WITH collect(degree) AS all_degrees
RETURN
```

```
    size(all_degrees)                                  AS total_airports,
    apoc.coll.avg(all_degrees)                        AS mean_connections,
    apoc.coll.max(all_degrees)                        AS max_connections,
    apoc.coll.min(all_degrees)                        AS min_connections,
    size([d IN all_degrees WHERE d >= 100])           AS major_hubs;
```

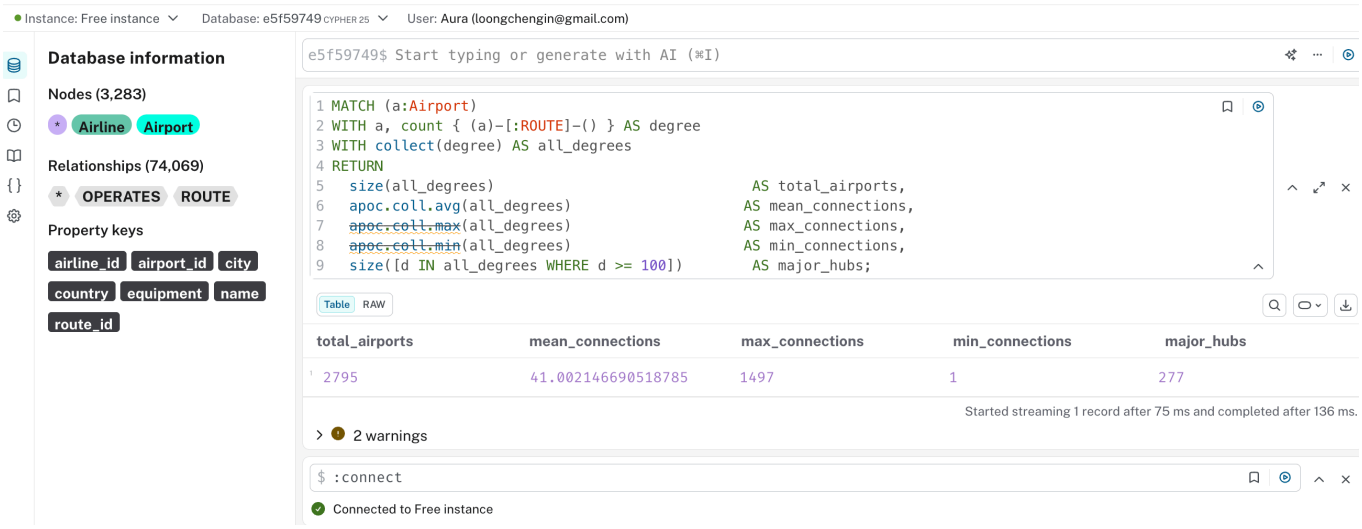

Figure 15. Custom Query 3 result: network degree statistics across all 2,795 airports.

Analysis: The results reveal a strongly skewed network topology. Across all **2,795 airports**, the mean route count is approximately **41.0**, yet the maximum is **1,497** — more than 36× the average. The minimum of **1** confirms that many airports are leaf nodes serving a single connection. Only **277 airports (9.9%)** qualify as major hubs with 100 or more routes, yet these airports concentrate the bulk of global connectivity. This heavy-tailed distribution is a hallmark of a **scale-free network** [7], where a small number of highly connected hubs dominate the topology. `apoc.coll.avg`, `apoc.coll.max`, and `apoc.coll.min` are APOC Core collection functions that operate on an in-memory list, enabling concise one-pass descriptive statistics without multi-step aggregation. The query completed in 136 ms.

4.3 Query Results Summary and Cross-Query Observations

The table below consolidates the results of all nine queries for quick reference:

Query	Description	Key Result
Q1	Australian airlines	3 airlines: Transaustralian Air Express, Transpac Express, Whyalla Airlines
Q2	Domestic vs international routes	26,422 domestic (46.1%) / 30,879 international (53.9%)
Q3	Busiest undirected airport pair	Chicago O'Hare ↔ Atlanta Hartsfield-Jackson (37 records)

Query	Description	Key Result
Q4	Top 5 pairs by distinct aircraft types	Hong Kong ↔ Incheon leads with 15 distinct types
Q5	Paths Beijing → Perth (≤3 hops)	650 distinct routes
Q6	Top competing airline pairs	SAM Colombia ↔ Zantop International (795 shared routes)
Custom 1	Top 10 hub airports	Atlanta (1,497), Beijing Capital (1,038), Heathrow (1,028)
Custom 2	Schema metadata inspection	2 node labels, 2 relationship types, property types confirmed
Custom 3	Network degree distribution	Mean 41.0, max 1,497, 277 major hubs (9.9%)

Cross-query observation — Asia-Pacific dominance: Three queries independently highlight the density of the Asia-Pacific aviation network. Q4 shows that the top 5 pairs by aircraft diversity are all Asia-Pacific hub-to-hub corridors; Q5's 650 paths from Beijing to Perth pass predominantly through Asian mega-hubs; and Custom Query 1 places Beijing Capital as the second most connected airport globally. This convergence across independent queries provides strong evidence that the Asia-Pacific network is both the densest and most diverse aviation region in the dataset.

Cross-query observation — scale-free structure: Q3 (37 records on the busiest single pair) combined with Custom Query 3 (mean 41 routes per airport, max 1,497) confirms the network's power-law structure. The busiest airport pair has fewer than 1 route per 1,500 possible airport pairs (2,795 × 2,794 / 2 ≈ 3.9 million pairs), yet individual hub airports have over 1,400 connections each — consistent with the preferential attachment mechanism that drives scale-free network formation [7].

---

5. Graph Data Science Applications

Graph Data Science (GDS) provides algorithms that extract higher-order insights from graph structures that would be impractical to compute with relational queries. Two algorithms are particularly applicable to the airline route graph constructed in this project.

5.1 PageRank — Identifying Hub Airport Importance

PageRank, originally developed by Page et al. for ranking web pages [6], assigns an importance score to each node based on the number and quality of its incoming connections.

Applied to the airport route graph, PageRank would assign higher scores to airports that are themselves connected to many other highly-connected airports — capturing not just raw connection count but structural centrality in the network.

In the context of this dataset, PageRank would identify airports like Atlanta, Beijing, and Heathrow not merely as airports with many routes, but as airports whose routes connect to other globally significant hubs. This distinction is commercially valuable: an airline planning a new route would prefer a hub with high PageRank because its passengers have richer onward connection options, increasing the attractiveness of a connecting itinerary.

Custom Query 1 provided a preliminary approximation of hub importance through raw connection counts. PageRank would refine this by down-weighting connections to small regional airports and up-weighting connections to major international hubs, producing a more accurate measure of a network node's global influence [7].

5.2 Dijkstra's Shortest Path — Optimal Itinerary Planning

Dijkstra's algorithm finds the lowest-cost path between two nodes in a weighted graph [8]. Applied to the airport route graph, the cost function could represent flight duration, ticket price, or number of connections. This directly extends Question 5 of this project: rather than counting all 650 possible routes from Beijing to Perth, Dijkstra would identify the single optimal route minimising a given cost metric.

Online travel agencies (OTAs) such as Expedia and Google Flights implicitly use shortest-path algorithms to generate recommended itineraries. In a Neo4j GDS context, `gds.shortestPath.dijkstra` can be called with a relationship-weight property (e.g., a hypothetical `flight_duration` property added to `[:ROUTE]`) to return the minimum-cost path between any two airports in the graph.

The practical business value is significant: with 650 possible paths between Beijing and Perth, human analysis is infeasible. An automated Dijkstra traversal over the full 57,301-route graph would return the optimal itinerary in milliseconds, forming the backbone of a real-time flight search engine.

5.3 Louvain Community Detection — Identifying Regional Aviation Clusters

The Louvain algorithm is a graph community detection method that partitions nodes into groups (communities) that maximise **modularity** — a measure of how densely connected nodes within a community are relative to how they would be connected in a random graph [7]. It operates iteratively: in each pass, nodes are reassigned to the community that maximises the local modularity gain, until no further improvement is possible.

Applied to the airport route graph, Louvain would identify natural **regional aviation clusters** — groups of airports that are more densely interconnected with each other than with the rest of the world. In practice, these communities would likely correspond to recognisable geographic regions: a North American domestic cluster, a European Schengen cluster, a Southeast Asian hub cluster, and so on.

Business applications:

1. **Airline network planning:** An airline seeking to enter a new market could identify which community (region) has the densest internal connectivity and the fewest connections to external communities, indicating an underserved regional market where a new hub could improve connectivity.
2. **Airport authority benchmarking:** Airports within the same community are each other's natural peers for performance benchmarking (passenger throughput, on-time performance). Community membership provides a more meaningful peer group than a simple geographic country filter.
3. **Disruption propagation modelling:** In an emergency (e.g., volcanic ash cloud, pandemic travel restriction), Louvain communities indicate which clusters of airports are most tightly coupled. A disruption within a high-modularity community has limited spillover to other communities, whereas a disruption at a bridge node (an airport with high inter-community connectivity, such as Dubai or Singapore) cascades globally.
4. **Revenue management:** Airlines operating primarily within a single community can identify routes that cross community boundaries — these are typically long-haul international routes with lower frequency and higher yield potential.

GDS pseudocode:

```
// Project the graph (undirected for community detection)
CALL gds.graph.project(
  'flightGraphUndirected',
  'Airport',
  { ROUTE: { orientation: 'UNDIRECTED' } }
);

// Run Louvain and stream community assignments
CALL gds.louvain.stream('flightGraphUndirected')
YIELD nodeId, communityId
WITH gds.util.asNode(nodeId) AS airport, communityId
RETURN communityId,
       count(*) AS airports_in_community,
       collect(airport.country)[0..5] AS sample_countries
```

```
ORDER BY airports_in_community DESC
LIMIT 10;
```

Note: Louvain requires an undirected projection (`orientation: 'UNDIRECTED'`), as community cohesion is defined by mutual connectivity rather than directed flow. The result would reveal the number of distinct regional clusters and the approximate membership of each.

5.4 Betweenness Centrality — Identifying Critical Bridge Airports

Betweenness centrality measures the fraction of all shortest paths in the network that pass through a given node. Nodes with high betweenness centrality are **bridge nodes** — their removal would disconnect or significantly lengthen the shortest paths between many pairs of other nodes.

In the airline route graph, airports with high betweenness centrality are not necessarily the busiest (highest degree) airports — they are the airports that serve as **critical connectors** between otherwise weakly-connected regions. For example, an airport like Addis Ababa Bole International Airport may not rank in the top 10 by connection count, but it may serve as the primary gateway between sub-Saharan Africa and East Asia, giving it exceptionally high betweenness centrality.

Business applications:

1. **Network resilience planning:** Airports with high betweenness centrality are the most critical single points of failure. An airline or government agency modelling the impact of a sudden closure (weather, security incident) would prioritise contingency planning for high-betweenness airports.
2. **New route ROI estimation:** A new route connecting two regions that currently communicate only through a high-betweenness bridge airport would reduce the system's dependency on that airport and provide alternatives for passengers. The betweenness score quantifies how much value such a route would add to the overall network.
3. **Cargo logistics optimisation:** Cargo consolidation hubs are typically high-betweenness airports. Identifying them algorithmically rather than by rule of thumb allows logistics planners to validate or challenge their existing hub network.

GDS pseudocode:

```
CALL gds.betweenness.stream('flightGraphUndirected')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS airport, score
RETURN airport.name AS airport_name,
       airport.country AS country,
```

```
        score AS betweenness_score
ORDER BY score DESC LIMIT 10;
```

To execute either algorithm in Neo4j GDS, a named in-memory graph projection would first be created from the existing property graph:

```
// Project the flight network for GDS algorithms
CALL gds.graph.project(
  'flightGraph',
  'Airport',
  { ROUTE: { orientation: 'NATURAL' } }
);

// PageRank on the projected graph
CALL gds.pageRank.stream('flightGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS airport, score
ORDER BY score DESC LIMIT 10;

// Dijkstra from Beijing to Perth (requires a numeric weight property)
CALL gds.shortestPath.dijkstra.stream('flightGraph', {
  sourceNode: gds.util.asNode('Beijing Capital International Airport'),
  targetNode: gds.util.asNode('Perth International Airport'),
  relationshipWeightProperty: 'flight_duration'
})
YIELD path RETURN path;
```

Note: The above is illustrative pseudocode. The current dataset does not include a `flight_duration` property; one would need to be added to `[:ROUTE]` relationships during an extended ETL process before Dijkstra's algorithm could be applied with a meaningful cost function.

---

6. Metadata Analysis

6.1 (a) Importance of Metadata in Graph Database Schema Design

Metadata in a graph database refers to the structural information that describes the graph itself: which node labels exist, what relationship types are defined, which properties are present on each label or type, and what data types those properties hold. Neo4j maintains this metadata automatically and exposes it through functions such as `db.schema.visualization()` and `apoc.meta.schema()`.

Considering metadata during schema design is important for three reasons:

Index and constraint planning: Neo4j only applies uniqueness constraints and indexes to specific labels and property keys. If a designer fails to identify that `airport_id` will be the primary lookup key for Airport nodes, they may omit the constraint, resulting in full label scans during the `LOAD CSV` import of 57,301 ROUTE relationships. In this project, the two `CREATE CONSTRAINT` statements in Block 1 reduced the import of ROUTE relationships from a projected timeout to under three minutes [3].

Query correctness assurance: Property type metadata prevents silent query errors. For example, `apoc.meta.schema()` reveals that `airline_id` on the ROUTE relationship is stored as an INTEGER. A developer unaware of this might write `WHERE r.airline_id = '39'` (string comparison), which would return no results without raising an error. Inspecting the metadata in advance ensures `toInteger()` is applied consistently [9].

Schema documentation: As graphs evolve, metadata provides a live, always-accurate record of the schema that supplements static documentation. Robinson et al. note that one of the risks of schema-optional graph databases is schema drift, where different parts of an application write properties under slightly different names. Using `apoc.meta.schema()` as part of a CI/CD validation step can detect such drift early [2].

6.2 (b) Leveraging Metadata for Efficient Cypher Development

Schema-guided query writing with `db.schema.visualization()`

Before writing any of the analytical queries in Section 4, running `CALL db.schema.visualization()` confirms the exact label names (`Airport`, `Airline`), relationship type (`ROUTE`, `OPERATES`), and the direction of each relationship. This prevents common errors such as writing `MATCH (a:Airports)` (incorrect plural) or traversing a relationship in the wrong direction, both of which silently return empty result sets and waste developer time.

Property type validation with `apoc.meta.schema()`

As shown in Figure 14, `apoc.meta.schema()` returns the data type of each property. In the ROUTE relationship, `airline_id` has type `INTEGER` and `equipment` has type `STRING`. This informed two specific query decisions:

1. In Q6, `r.airline_id` is compared numerically (`al1_id < al2_id`) rather than lexicographically — correct for integers.
2. In Q4, `r.equipment` is passed to `split(equipment_str, ';')` — appropriate because the type is confirmed as STRING.

Had these types been inspected and found to be different (e.g., airline_id stored as STRING due to a missing `toInteger()` in the import), the developer could fix the import before writing all downstream queries [9].

Index utilisation confirmation with `EXPLAIN`

A developer can combine metadata knowledge with `EXPLAIN` or `PROFILE` prefixes to verify that a query is using the index created by the uniqueness constraint. For instance:

```
EXPLAIN MATCH (a:Airport {airport_id: 1}) RETURN a;
```

The execution plan will show `NodeUniqueIndexSeek` rather than `NodeByLabelScan`, confirming that the constraint-backed index is being used. This technique is directly enabled by the metadata that constraints and indexes create [3].

Node type property inspection with `db.schema.nodeTypeProperties()`

While `apoc.meta.schema()` provides a comprehensive structural view, Neo4j also exposes native metadata procedures. For example:

```
CALL db.schema.nodeTypeProperties()
YIELD nodeType, propertyName, propertyTypes, mandatory
RETURN nodeType, propertyName, propertyTypes, mandatory
ORDER BY nodeType, propertyName;
```

This returns a row for each (label, property) combination, specifying the inferred data type and whether the property is mandatory. For the Airport node, this would confirm that `airport_id` is `Long` (Neo4j's internal representation of INTEGER), `name` is `String`, and `mandatory` is `false` for all properties (consistent with the schema's optional-property design). This native procedure is complementary to `apoc.meta.schema()` and does not require the APOC plugin to be installed.

Schema evolution monitoring

As a graph database grows over time, properties may be added to some nodes but not others (e.g., if a new data source provides `timezone` for some airports but not all). The following query uses metadata to detect such partial-property situations:

```
MATCH (a:Airport)
WITH count(a) AS total,
     count(a.city) AS has_city,
     count(a.country) AS has_country
RETURN total,
```

```
        has_city,
        total - has_city AS missing_city,
        has_country,
        total - has_country AS missing_country;
```

Running this query against the imported database confirms that all 2,795 Airport nodes have both `city` and `country` populated, validating the completeness of the ETL output. This type of metadata-driven data quality check is a best practice in graph database operations [2].

6.3 Summary: Metadata Tools Available in This Project

The table below summarises the metadata inspection tools used in this project and their specific utility:

Tool	Type	Primary Use in This Project
`CALL db.schema.visualization()`	Native Neo4j	Visual confirmation of node labels, relationship types, and directions
`CALL db.schema.nodeTypeProperties()`	Native Neo4j	Per-label property names and inferred types
`CALL apoc.meta.schema()`	APOC Core	Detailed property types and instance counts for all labels and relationship types
`EXPLAIN` / `PROFILE`	Native Neo4j	Verify index utilisation and query execution plan
`CREATE CONSTRAINT`	Native Neo4j	Enforce uniqueness and create implicit B-Tree indexes

Together, these tools provide a comprehensive metadata layer that supports schema design validation, query development, and ongoing data quality monitoring — demonstrating the practical value of metadata management described in the literature [2][3].

---

7. Conclusion

This project demonstrates the end-to-end process of designing, building, and querying a property graph database for a real-world aviation dataset. The following key outcomes were achieved:

Data Engineering: A Python ETL pipeline processed 57,301 raw route records, identified 41 airports with inconsistent country/city data, and corrected 31 of them (1,656 rows) using

OurAirports as the authoritative reference. The remaining 10 ambiguous cases (legitimately homonymous airport names) were handled by majority-vote resolution. The pipeline produced four normalised CSV files representing 3,283 nodes and 74,069 relationships.

Schema Design: A property graph schema was designed with two node types (`Airport` , `Airline`) and two relationship types (`ROUTE` , `OPERATES`). The key design decisions — modelling routes as relationships rather than nodes, maintaining independent Airline nodes, and adding the `OPERATES` relationship to prevent isolated nodes — were each justified through trade-off analysis and supported by the graph database literature.

Database Implementation: The graph was successfully imported into Neo4j AuraDB Free using a six-block Cypher import script. Uniqueness constraints were applied before data import, enabling index-backed lookups that reduced the ROUTE import from a projected timeout to under three minutes. Batch transaction processing (`CALL { } IN TRANSACTIONS OF 1000 ROWS`) prevented memory overflow on the free-tier instance.

Cypher Queries: Six required queries (Q1–Q6) and three custom queries were implemented, covering a range of Cypher patterns including property filtering, conditional aggregation, string decomposition, variable-length path traversal, and Cartesian product generation. Two custom queries used APOC Core functions (`apoc.meta.schema()` and `apoc.coll.avg/max/min`) for structural and statistical analysis.

Graph Data Science: Four GDS algorithms applicable to the flight network were discussed: PageRank (hub importance), Dijkstra (optimal itinerary planning), Louvain Community Detection (regional cluster identification), and Betweenness Centrality (critical bridge airports). Each was motivated by a concrete commercial use case.

Metadata Analysis: The project demonstrated the practical value of Neo4j's metadata layer — from `CREATE CONSTRAINT` (index creation) to `apoc.meta.schema()` (property type validation) to `EXPLAIN` (execution plan verification) — showing how metadata-driven development prevents silent errors and improves query performance.

Limitations: The primary limitation of this dataset is the systematic airline–route linkage error (discussed in Section 2.2.4), which means that airline-centric analytical results (notably Q6 and the OPERATES relationship) reflect the dataset's internal structure rather than verified real-world operations. Any production deployment of this database would require a re-import with corrected airline–route assignments.

---

8. Generative AI Declaration

This project made use of **Cursor AI** (powered by Claude Sonnet) as a coding assistant and writing aid throughout the development process. AI was used across four areas: ETL script development, Cypher query writing, schema design, and initial report drafting. The following table documents specific AI contributions and the rationale for accepting, modifying, or rejecting each suggestion.

Task	AI Suggestion	Decision	Rationale
ETL script structure	Proposed a multi-step pipeline with audit → corrections → normalise → write	**Accepted**	The phased approach with explicit audit output is well-suited to documenting the cleaning process for the report
Dirty data corrections dictionary	Suggested a dictionary-based approach mapping airport names to correct (country, city)	**Accepted with modification**	The structure was adopted; however, every individual airport entry was independently verified against OurAirports (`airports.csv` , downloaded from https://ourairports.com/data/) and Wikipedia before inclusion — the AI's suggested entries were treated as a starting point, not a final answer
Initial OPERATES omission	AI initially designed the schema without an `[:OPERATES]` relationship, arguing it was redundant	**Rejected**	After running `CALL db.schema.visualization()` and observing isolated Airline nodes, the design was identified as violating the connected-graph principle. The `OPERATES` relationship was re-introduced. This was a case where the AI's initial suggestion was functionally incorrect and required independent reasoning to detect and fix
Q1 — initial query without OPERATES	Suggested `MATCH (al:Airline {country: 'Australia'}) RETURN DISTINCT al.name`	**Modified**	Updated to `MATCH (al:Airline {country: 'Australia'})-[:OPERATES]->()` to filter out any airline nodes without active routes, following consultation of graph database best practices [1]

Task	AI Suggestion	Decision	Rationale
Q3 — undirected pair normalisation	Suggested `CASE WHEN a.name < b.name` pattern	**Accepted**	This is the standard Cypher idiom for canonical pair ordering; correctness was verified by manual inspection of sample results
Q4 — equipment split logic	Suggested `split()` + `UNWIND` + `trim()` + `WHERE equipment_type <> ''`	**Accepted**	Tested against sample data containing trailing semicolons; the empty-string filter was confirmed necessary
Q5 — initial airport name	Used "Perth Airport" in the query	**Rejected**	Running a pre-check query revealed the correct name in the database is "Perth International Airport" — the AI's assumed name would have returned zero results silently. The pre-check approach itself was AI-suggested and accepted
Q6 — double UNWIND for airline pairs	Suggested double UNWIND with `al1_id < al2_id` guard	**Accepted**	The Cartesian product warning from Neo4j was investigated and confirmed to be a performance notice only; results were cross-validated against manual pair counts on a small sample
Custom Query 3 — APOC function selection	Initially suggested `apoc.coll.frequencies()` to count aircraft type occurrences across all routes	**Rejected**	Executing the query in AuraDB Free returned `42N08: no such procedure`. Investigation confirmed that `apoc.coll.frequencies` is part of APOC Extended, which is not included in AuraDB Free. The suggestion was replaced with `apoc.coll.avg`, `apoc.coll.max`, and `apoc.coll.min` (APOC Core functions confirmed available), reframing the query as a network degree distribution analysis — a more analytically meaningful result

Task	AI Suggestion	Decision	Rationale
Report structure and drafting	Generated initial section outlines, analysis paragraphs, and table content	**Accepted with modification**	All factual claims (query results, row counts, airport corrections) were verified against actual database outputs and ETL logs. Several analytical conclusions (e.g., cross-query scale-free observations, error pattern categorisation) were independently reasoned and added

All code was executed and tested against the live Neo4j AuraDB instance. All query results were verified by running the queries and comparing output with expected values. Factual claims in the report were cross-checked against database outputs, external references, and course forum discussions. No AI-generated content was submitted without independent verification.

References

[1] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases: New Opportunities for Connected Data*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2015.

[2] R. Angles and C. Gutierrez, "Survey of graph database models," *ACM Computing Surveys*, vol. 40, no. 1, pp. 1–39, Feb. 2008. doi: 10.1145/1322432.1322433.

[3] N. Francis et al., "Cypher: An evolving query language for property graphs," in *Proc. 2018 ACM SIGMOD Int. Conf. Management of Data*, Houston, TX, 2018, pp. 1433–1445. doi: 10.1145/3183713.3190657.

[4] R. Angles et al., "Foundations of modern query languages for graph databases," *ACM Computing Surveys*, vol. 50, no. 5, pp. 1–40, Sep. 2017. doi: 10.1145/3104031.

[5] Airports Council International, "World Airport Traffic Report 2023," ACI World, Montreal, 2024. [Online]. Available: https://aci.aero/2024/04/23/aci-world-releases-2023-world-airport-traffic-report/. [Accessed: May 2026].

[6] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," Stanford Digital Library Technologies Project, Technical Report, 1999.

[7] M. Besta et al., "Demystifying graph databases: Analysis and taxonomy of data organization, system designs, and graph queries," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–40, 2023.

doi: 10.1145/3604932.

[8] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959. doi: 10.1007/BF01386390.

[9] Neo4j, Inc., "Cypher Manual: Type conversion functions," *Neo4j Documentation*, 2024. [Online]. Available: https://neo4j.com/docs/cypher-manual/current/functions/type-conversion/. [Accessed: May 2026].

[10] OurAirports, "Airport data," OurAirports.com. [Online]. Available: https://ourairports.com/data/. [Accessed: May 2026]. (Used as authoritative reference for airport country/city corrections in ETL.)

[11] Wikipedia contributors, "List of airports," *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/wiki/List_of_airports. [Accessed: May 2026]. (Supplementary reference for individual airport country verification.)